Week 12 Technical Report - Fast Transforms, Numerical Stability, and Signal and Image Processing

Project repository (complete source, tests, benchmarks and results):
https://github.com/RDeibel2025/CTX-Advanced-Algorithms
Fast transforms: src/fft_analyzer/ · demonstration: examples/week12_signal_demo.py
Benchmark: benchmarks/week12_benchmark.py

Robert Deibel · CSC 5300 Advanced Algorithms · Concordia University Texas · Fall 2026

📎 Complete project repository

https://github.com/RDeibel2025/CTX-Advanced-Algorithms

1. Executive Summary

I built an FFT analyzer: a direct DFT, recursive and iterative radix-2 Cooley-Tukey FFTs, classical, blocked and Strassen matrix multiplication, LU with partial pivoting, three QR methods, and signal, image and audio modules. Every module follows one pattern: move the problem to where it is easy, do the easy thing, move back, and measure what the round trip cost. At N = 4,096 the iterative FFT ran 268 times faster than the direct DFT, and at N = 2^20 its round-trip error was 6.26e-16. Solve error stayed below cond(H)·eps on Hilbert matrices up to n = 12. Classical Gram-Schmidt lost orthogonality completely by n = 7 while Householder stayed below 2.2e-15. Quality 50 compression kept 16.4% of the DCT coefficients at 28.86 dB.

2. Numerical Algorithm Theory

The DFT evaluates a degree N - 1 polynomial at the N complex roots of unity, which costs O(N^2). Cooley-Tukey splits the input into even and odd samples, solves two half-size transforms, and combines them with N/2 twiddle multiplications, so T(N) = 2T(N/2) + O(N) = O(N log N) (Cooley & Tukey, 1965; CLRS Ch. 30). The iterative version reorders the input by bit reversal and runs log2 N butterfly stages in place. Convolution in time is pointwise multiplication in frequency, so convolving two signals is an FFT, a multiply and an inverse FFT (CLRS Ch. 30; Amakobe, 2025, Ch. 12).

Linear algebra makes the same move. LU with partial pivoting factors PA = LU once, after which each solve is two O(n^2) triangular substitutions (CLRS Ch. 28). QR factors A into an orthogonal Q and a triangular R. Strassen multiplies 2 x 2 block matrices with seven products instead of eight, giving O(n^(log2 7)) = O(n^2.81) (Strassen, 1969). Each round trip adds rounding error. The FFT's error grows like eps·log2 N. A backward-stable solve has relative error up to about cond(A)·eps, and the orthogonality of a computed Q depends on how it was built (Golub & Van Loan, 2013).

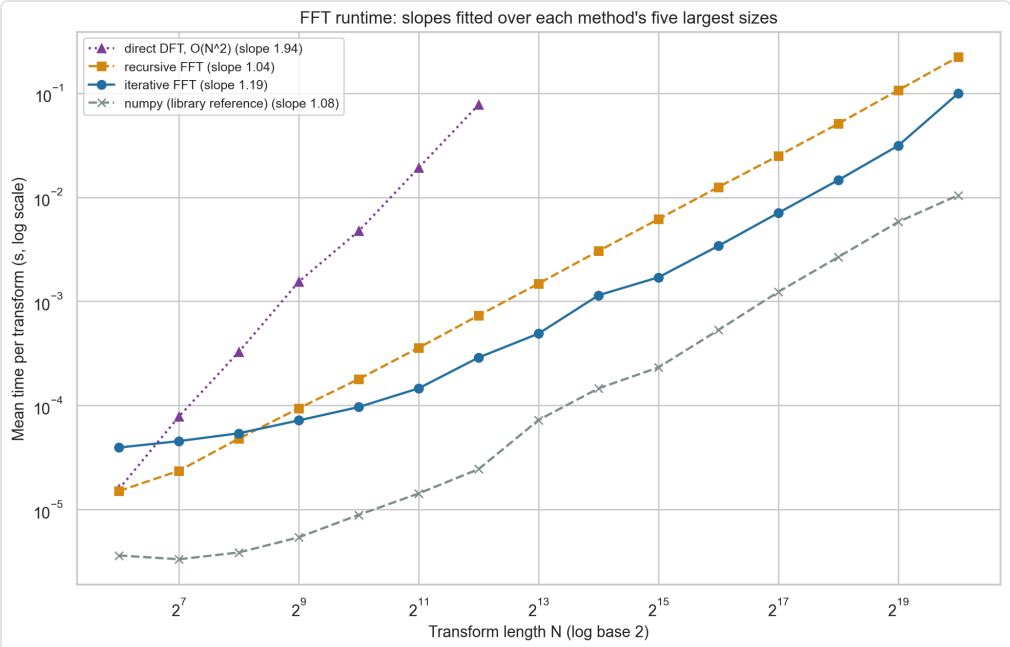
3. Methodology

Test conditions. Apple M2 Max, 64 GB RAM, macOS 14.5 (arm64), CPython 3.12.4, numpy 1.26.4. Times come from `time.perf_counter`. Each method and size gets one untimed warm-up call, then 3 timed runs. In the multiply study, sizes above 256 get one timed run, and numpy's `@` gets 5 at every size. A SIGALRM cap stops any run past 20 s.

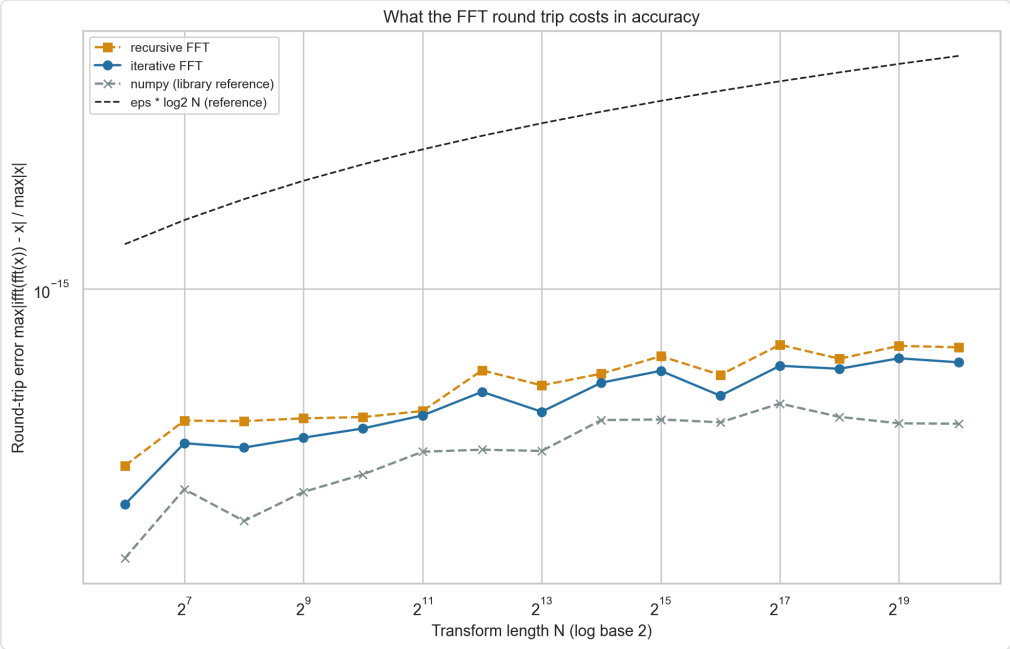
Inputs. The master seed is 42, and every random input comes from `derive_seed(42, experiment, n, rep)`, which gives 22 distinct seeds. FFT inputs are complex standard normal vectors, N = 2^6 to 2^20, with the DFT run to 2^12. Matrices are standard normal, n = 32 to 1,024. The classical triple loop was capped above n = 256 by configuration, so n = 512 and 1,024 are marked as not run. Stability uses Hilbert matrices, n = 2 to 12. The image, the audio file and the time series are synthetic: a 256 x 256 gradient with a disc, a checkerboard and noise; two tones (440 Hz and 3,000 Hz) plus white noise; and a trend with a weekly cycle.

References. numpy's `np.fft.fft`, `@` and `np.linalg.cond` appear only as checks and as labelled reference rows. No library FFT or linear-algebra solver is called inside `src/fft_analyzer`. Every FFT and multiply result was compared with numpy's, and the largest relative error, max|a - b| / max|b|, was 9.8e-15.

4. FFT and Spectral Analysis

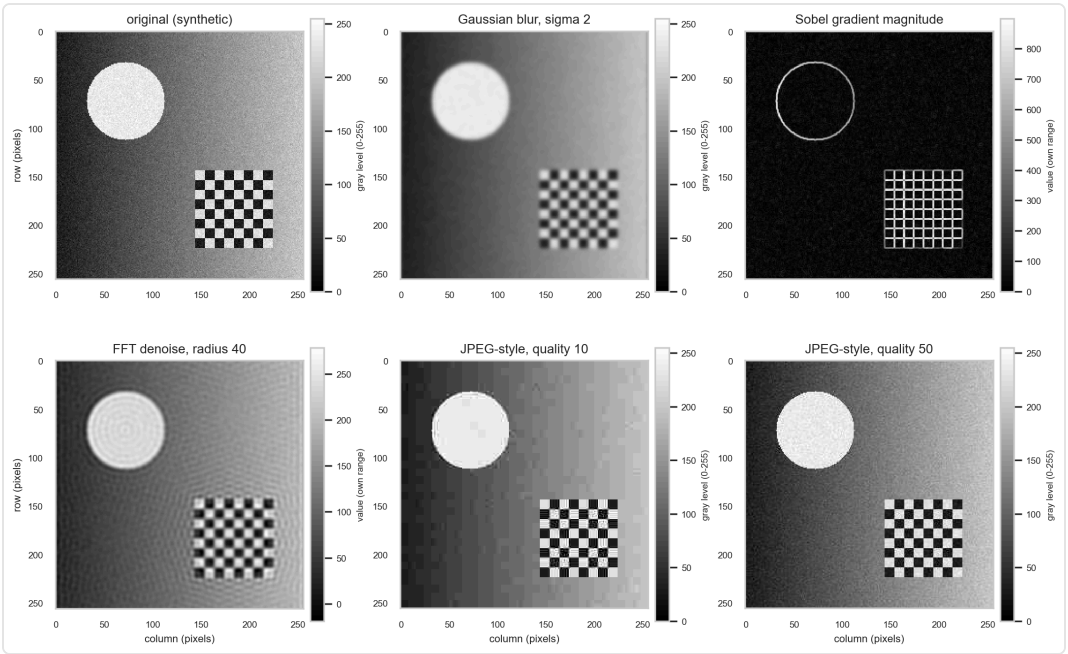


Over each method's five largest sizes, the DFT's log-log slope was 1.94, close to the predicted 2, and the recursive FFT's was 1.04. The iterative FFT's was 1.19, because its time tripled from 2^19 (31.4 ms) to 2^20 (99.6 ms); I did not measure why. At 2^20 the recursive FFT took 222.9 ms. The recursive version was faster up to N = 2^8, and the iterative one from 2^9 on. numpy's compiled FFT (library reference) was 9.5 times faster than the iterative FFT at 2^20.

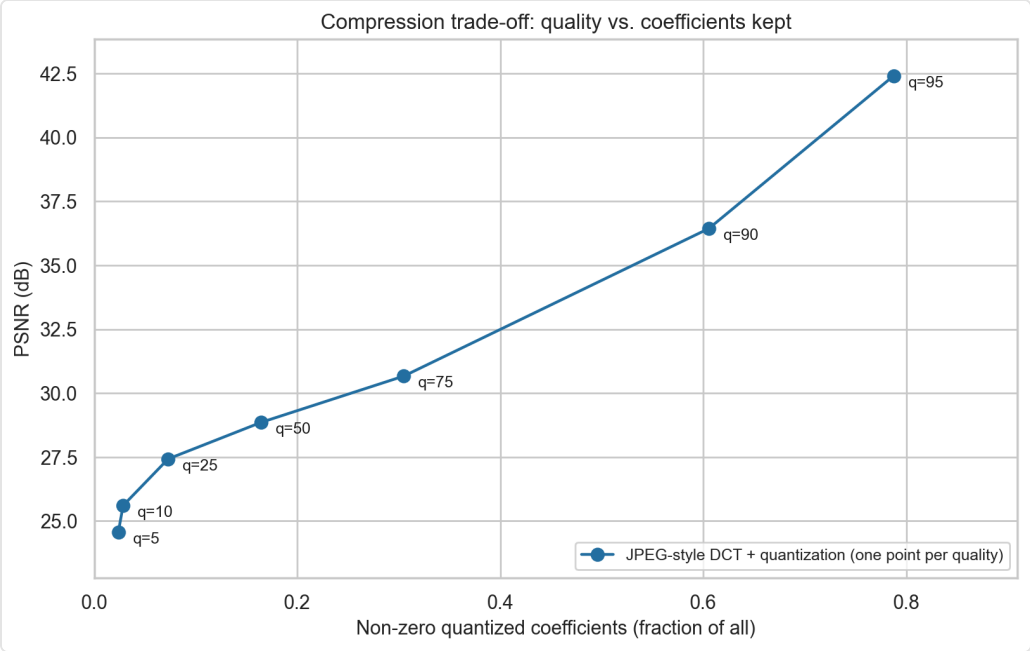


The round trip ifft(fft(x)) cost almost nothing in accuracy. The iterative error ranged from 2.52e-16 to 6.42e-16, or 0.14 to 0.24 of eps·log2 N, and the recursive error was 0.16 to 0.28 of that bound. The benchmark's spectrogram shows a 100 to 400 Hz chirp; in the signal demo's chirp the dominant frequency rose from 109.4 Hz to 390.6 Hz and never fell after 3-frame smoothing. On the two-tone test signal, a low-pass at 80 Hz left 7.94e-6 of the 120 Hz tone's energy, and a high-pass left 3.61e-8 of the 50 Hz tone's. The low-pass misses the specification's 1e-6 target over the full record: the filter zero-pads 1,000 samples to 1,024, and the brick wall rings near both ends. On the interior (50 ms dropped at each end) the signal demo measures 2.6e-10. The filtering and FFT stages animations show these steps frame by frame.

5. Image and Audio Results



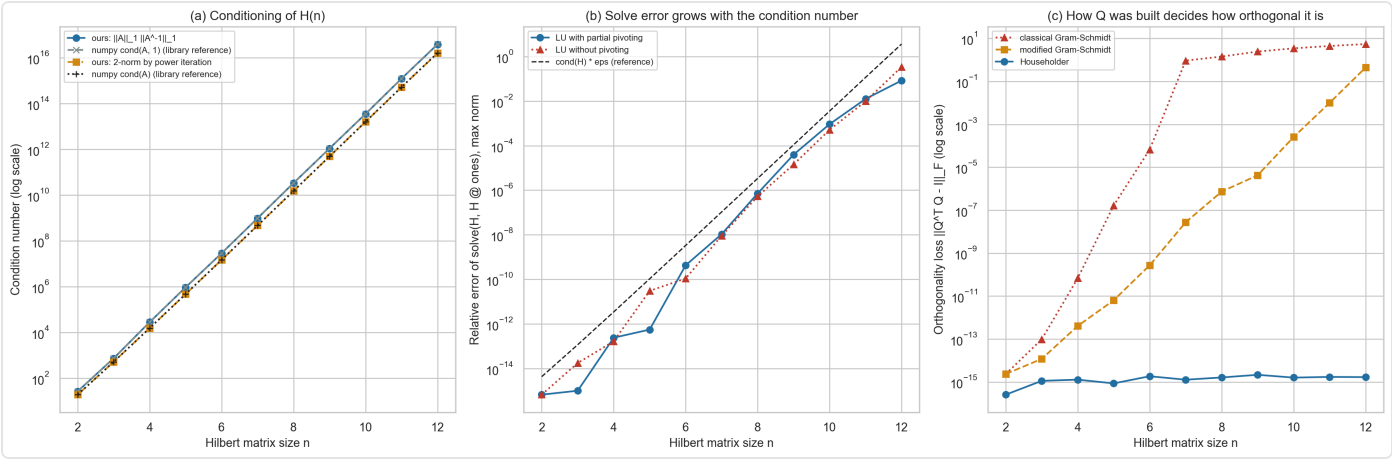
Blur and Sobel are circular convolutions computed through the 2-D FFT, and the tests match that convolution to `scipy.ndimage.convolve` with wrap mode within 1e-9. Denoising zeroes the high-frequency bins. Compression moves each 8 x 8 block into the DCT domain, divides by a JPEG quantization table scaled by quality, rounds, and transforms back (Wallace, 1992).



Quality 5 kept 2.36% of the coefficients at 24.57 dB, quality 50 kept 16.42% at 28.86 dB, and quality 95 kept 78.74% at 42.40 dB. From quality 75 to 95, PSNR rose 11.74 dB while the coefficients kept rose from 30.48% to 78.74%. The nonzero ratio stands in for file size; I did not implement entropy coding.

In the audio demo, a 1,000 Hz low-pass on the synthetic WAV left the 440 Hz peak at 0.999997 of its magnitude and the 3,000 Hz peak at 0.000047 (output WAV). Writing it as 16-bit PCM changed no sample by more than 1.53e-5, under 1/32768. On a 364-day synthetic series, the detrended spectrum peaked at bin 52, a period of 7.0 days, and the autocorrelation peaked at lag 7.

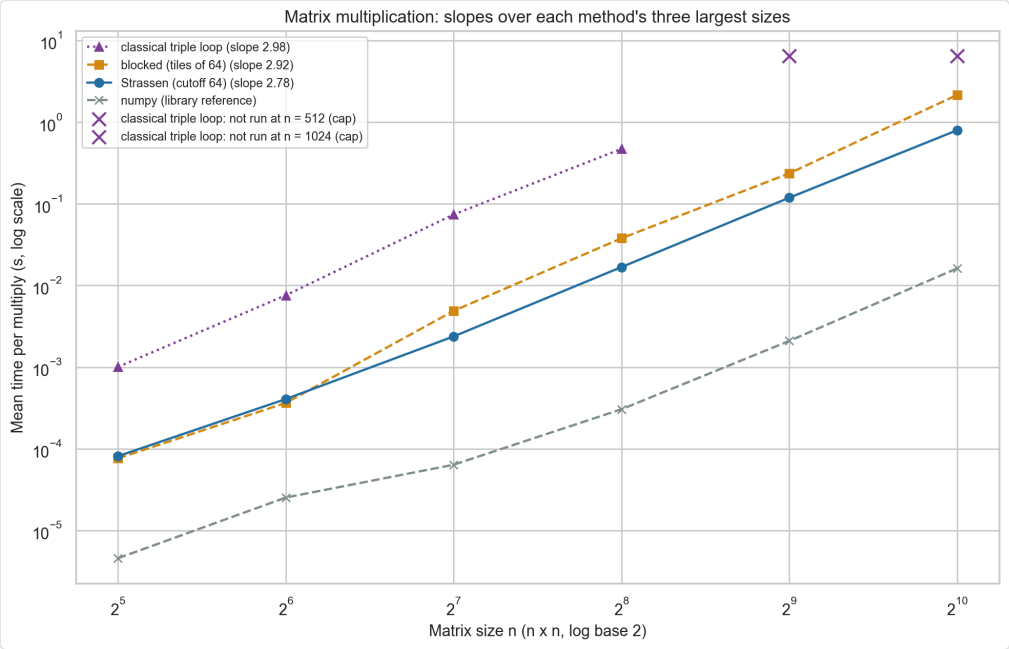
6. Stability Considerations



My 1-norm and power-iteration 2-norm condition numbers matched numpy's within a relative 2.1e-5 through n = 10, and differed by 3.5% and 2.5% at n = 12, where numpy's 2-norm value is 1.64e16. The pivoted solve's error grew from 6.66e-16 at n = 2 to 8.36e-2 at n = 12 and stayed between 0.01 and 0.36 of cond(H)·eps. Without pivoting, the error was smaller at seven of the eleven sizes and larger at n = 3, 5 and 12 (3.40e-1 against 8.36e-2). Hilbert matrices are symmetric positive definite, and elimination without pivoting is stable on such matrices (Golub & Van Loan, 2013). The test case [[1e-20, 1], [1, 1]] x = [1, 2] shows the failure pivoting prevents: without it the solve returns [0, 1], and with it [1, 1].

Orthogonality loss ||QᵀQ - I||_F separated the three QR methods. Classical Gram-Schmidt reached 0.927 at n = 7 and 5.48 at n = 12. Modified Gram-Schmidt reached 0.439 at n = 12, staying between 0.04 and 0.55 of cond(H)·eps. Householder never exceeded 2.15e-15.

7. Benchmark Results



The classical triple loop's slope was 2.98 and it took 0.471 s at n = 256. Blocking with 64 x 64 tiles was 12.4 to 20.8 times faster at n = 32 to 256, with a slope of 2.92, and took 2.157 s at n = 1,024. Strassen's slope was 2.78, near log2 7 = 2.81, and it took 0.797 s at n = 1,024. Its error was 9.8e-15 against 2.5e-15 for blocking. numpy's `@` (library reference) took 0.0162 s at n = 1,024.

8. Discussion

Convolution through the frequency domain costs O(N log N) against O(N^2) for the direct sum, and the measured round trip added error below eps·log2 N. LU's error followed the condition number. Strassen's speed came with about four times blocking's error. Householder kept Q orthogonal at every tested condition number, modified Gram-Schmidt lost orthogonality in step with cond·eps, and classical Gram-Schmidt had lost it entirely by n = 7, where cond(H) is 4.75e8. Compression is the one round trip that discards information on purpose, and the quality setting chooses how much.

9. Conclusion

The FFT turned O(N^2) work into O(N log N) at an accuracy cost of under 7.1e-16 in the round trip. LU, QR and the DCT each move a problem into a form where the work is simple, and the measurements show what each round trip costs: error that tracked cond·eps and stayed below it for the pivoted solve and for modified Gram-Schmidt, almost none for Householder, and a chosen loss of detail for compression.

References

- Amakobe, M. (2025). *Advanced Algorithms: Concepts, Complexity, and Applied Projects*, Ch. 12.
- Cooley, J. W., & Tukey, J. W. (1965). An algorithm for the machine calculation of complex Fourier series. *Mathematics of Computation, 19*(90), 297-301.
- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to Algorithms* (4th ed.). MIT Press. Ch. 28, Matrix Operations, and Ch. 30, Polynomials and the FFT.
- Golub, G. H., & Van Loan, C. F. (2013). *Matrix Computations* (4th ed.). Johns Hopkins University Press.
- Strassen, V. (1969). Gaussian elimination is not optimal. *Numerische Mathematik, 13*(4), 354-356.
- Wallace, G. K. (1992). The JPEG still picture compression standard. *IEEE Transactions on Consumer Electronics, 38*(1), xviii-xxxiv.

AI use

Claude (Claude Code) drafted the code, tests and this report's first draft from a specification I wrote from the instructions and required reading, including the anchor values the tests assert. Every measurement comes from running the benchmark on my machine; every benchmark result in this report is printed from the CSV by `tools/week12_facts.py`, and the application results come from the three demos in `examples/`. Full disclosure: `docs/AI_USE.md`.